

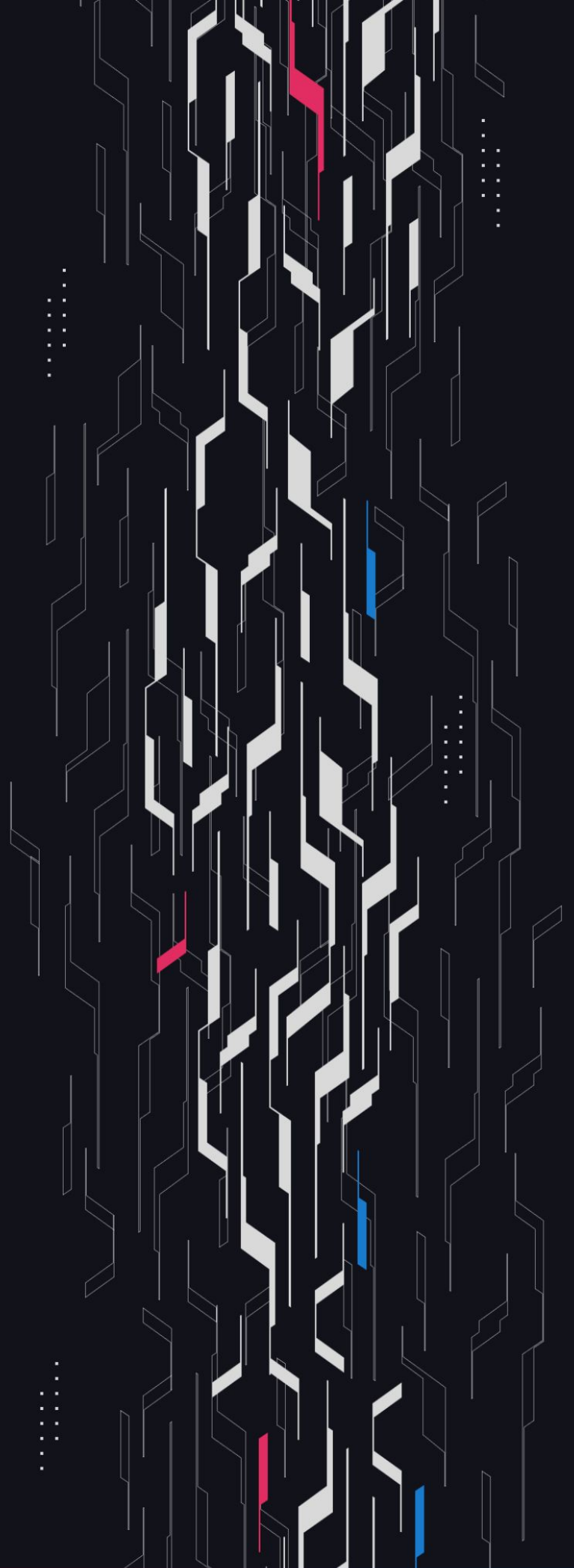
GA GUARDIAN

GMX

Risk Oracle

Security Assessment

May 26th, 2026



Summary

Audit Firm Guardian

Prepared By Nicholas Chew, Cosine

Client Firm GMX

Final Report Date May 26, 2026

Audit Summary

GMX engaged Guardian to perform a targeted review of their Risk Oracle updates. From May 18, 2026 to May 22, 2026, Guardian reviewed the incremental changes introducing the RISK_ORACLE role, with focus on on-chain permissions, access control flows, and market parameter update mechanisms supporting the migration from the external ChaosLabs Risk Oracle service to GMX-operated Risk Oracle infrastructure.

Confidence Ranking

Given the minimal High and Critical issues detected during the main review, Guardian assigns a Confidence Ranking of 4 to the protocol. Guardian advises the protocol to consider periodic review with future changes. For detailed understanding of the Guardian Confidence Ranking, please see the rubric on the following page.

✓ Verify the authenticity of this report on Guardian's GitHub: <https://github.com/guardianaudits>

Guardian Confidence Ranking

Confidence Ranking	Definition and Recommendation	Risk Profile
5: Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4: High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. Recommendation: Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3: Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas (e.g., weak modularity, risky patterns). No critical design flaws, though some patterns could lead to issues in edge cases. Recommendation: Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2: Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. Recommendation: Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week. Or additional High/Critical findings uncovered in remediation review which have not been resolved and confirmed by Guardian.
1: Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (≥ 5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. Recommendation: Halt deployment and seek a comprehensive re-audit after substantial refactoring.	≥ 5 High/Critical findings and overall systemic flaws.

Table of Contents

Project Information

Project Overview 5

Audit Scope & Methodology 6

Smart Contract Risk Assessment

Findings & Resolutions 9

Addendum

Disclaimer 25

About Guardian 26

Project Overview

Project Summary

Project Name	GMX
Codebase	https://github.com/gmx-io/gmx-synthetics-private
Commit(s)	Main Review: c49ebdea0001126908ad266dec1b870c254f2c6e Remediation Review: 3110433f4e61acff89a967e3107e9380d1221271

Audit Summary

Delivery Date	May 26, 2026
Audit Methodology	Static Analysis, Manual Review

Vulnerability Summary

Vulnerability Level	Total	Pending	Declined	Acknowledged	Partially Resolved	Resolved
● Critical	0	0	0	0	0	0
● High	0	0	0	0	0	0
● Medium	3	0	0	0	0	3
● Low	4	0	0	0	0	4
● Info	7	0	0	2	0	5

Audit Scope & Methodology

- Targeted security assessment of the incremental update introducing the RISK_ORACLE role to GMX mainnet ahead of the planned v2.3 release.
- Review of the newly introduced on-chain permissions, access control flows, and market parameter update mechanisms associated with the RISK_ORACLE role.
- Assessment limited to the isolated changes required to support migration from the external ChaosLabs Risk Oracle service to GMX's internally operated Risk Oracle infrastructure.
- Evaluation conducted with consideration for preserving the existing v2.2c deployment freeze and minimizing impact to unrelated system components.

Audit Scope & Methodology

Vulnerability Classifications

Severity	Impact: <i>High</i>	Impact: <i>Medium</i>	Impact: <i>Low</i>
Likelihood: <i>High</i>	● Critical	● High	● Medium
Likelihood: <i>Medium</i>	● High	● Medium	● Low
Likelihood: <i>Low</i>	● Medium	● Low	● Low

Impact

- High** Significant loss of assets in the protocol, significant harm to a group of users, or a core functionality of the protocol is disrupted.
- Medium** A small amount of funds can be lost or ancillary functionality of the protocol is affected. The user or protocol may experience reduced or delayed receipt of intended funds.
- Low** Can lead to any unexpected behavior with some of the protocol's functionalities that is notable but does not meet the criteria for a higher severity.

Likelihood

- High** The attack is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount gained or the disruption to the protocol.
- Medium** An attack vector that is only possible in uncommon cases or requires a large amount of capital to exercise relative to the amount gained or the disruption to the protocol.
- Low** Unlikely to ever occur in production.

Audit Scope & Methodology

Methodology

Guardian is the ultimate standard for Smart Contract security. An engagement with Guardian entails the following:

- Two competing teams of Guardian security researchers performing an independent review.
- A dedicated fuzzing engineer to construct a comprehensive stateful fuzzing suite for the project.
- An engagement lead security researcher coordinating the 2 teams, performing their own analysis, relaying findings to the client, and orchestrating the testing/verification efforts.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross-referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.
Comprehensive written tests as a part of a code coverage testing suite.
- Contract fuzzing for increased attack resilience.

Round One Findings & Resolutions

ID	Title	Category	Severity	Status
M-01	Funding/Borrowing Update Influence Stale Markets	Configuration	● Medium	Resolved
M-02	RiskOracleConfig Is Not Granted CONTROLLER Role	Configuration	● Medium	Resolved
L-01	Missing Cross-Parameter Sanity Checks	Configuration	● Low	Resolved
L-02	Risk Oracle Config Accepts Non-Market Addresses	Configuration	● Low	Resolved
L-03	Unnecessary Setters Exposed	Warning	● Low	Resolved
L-04	RISK_ORACLE Role Missing In Scripts	Warning	● Low	Resolved
I-01	Unused Imports	Superfluous Code	● Info	Resolved
I-02	_validateKey Performs A Redundant Role Check	Access Control	● Info	Resolved

M-01 | Funding/Borrowing Update Influence Stale Markets

Category	Severity	Location	Status
Configuration	● Medium	RiskOracleConfig.sol	Resolved

Description

RiskOracleConfig allows the RISK_ORACLE role to directly update funding and borrowing rate parameters without first settling the market’s accrued funding and borrowing state. Since the next market update calculates elapsed time from the previous update timestamp using the current config, a newly set rate can be applied to the entire unsettled interval.

This can overcharge or undercharge open positions and distort claimable funding or borrowing fees and therefore lead to step-wise jumps in positions health. This can even lead to positions being suddenly liquidatable. The more stale a market is the bigger the impact of such config changes.

Recommendation

Consider to update the market state (accrue funding, borrowing, etc.) before changing configurations that have any influence on a stale market state.

Resolution

GMX Team: Resolved.

M-02 | RiskOracleConfig Is Not Granted CONTROLLER Role

Category	Severity	Location	Status
Configuration	● Medium	RiskOracleConfig.sol, deployRiskOracleConfig.ts	Resolved

Description

RiskOracleConfig requires the CONTROLLER role to function because it writes through DataStore and emits through EventEmitter, both of which are onlyController-gated.

The deploy script deployRiskOracleConfig.ts attempts to grant CONTROLLER after deployment:

```
afterDeploy: async ({ deployedContract }) => {
  await grantRoleIfNotGranted(deployedContract, "CONTROLLER");
},
```

However, grantRoleIfNotGranted immediately returns on existing mainnet deployment networks:

```
if (hre.gmx.isExistingMainnetDeployment) {
  return;
}
```

Affected flows include:

- setRiskOracleMarketEnabled, because it calls DataStore.setBool.
- setBool, setAddress, setBytes32, setUint, and setInt, because they call the corresponding DataStore.set* method.
- All successful setter flows, because they emit through EventEmitter, which also requires CONTROLLER

Recommendation

Add RiskOracleConfig to requiredRolesForContracts.CONTROLLER in config/roles.ts, and ensure the mainnet role update transaction grants the deployed RiskOracleConfig address the CONTROLLER role.

Resolution

GMX Team: Resolved.

L-01 | Missing Cross-Parameter Sanity Checks

Category	Severity	Location	Status
Configuration	● Low	RiskOracleConfig.sol	Resolved

Description

RiskOracleConfig allows the RISK_ORACLE role to directly update sensitive market parameters. Right now, values are mostly checked against broad per-key bounds, but important cross-parameter invariants are not enforced. For example, MIN_COLLATERAL_FACTOR should remain greater than or equal to MIN_COLLATERAL_FACTOR_FOR_LIQUIDATION; otherwise users may be able to open or increase positions that are immediately liquidatable.

Recommendation

Consider to add invariant checks before applying risk oracle updates.

Resolution

GMX Team: Resolved.

L-02 | Risk Oracle Config Accepts Non-Market Addresses

Category	Severity	Location	Status
Configuration	● Low	RiskOracleConfig.sol: 57-73	Resolved

Description

RiskOracleConfig.setRiskOracleMarketEnabled only checks that the given market != address(0) before setting RISK_ORACLE_MARKET_ENABLED. It does not verify that the supplied address is a registered GMX market.

Recommendation

Consider to validate that the given market exists in GMX.

Resolution

GMX Team: Resolved.

L-03 | Unnecessary Setters Exposed

Category	Severity	Location	Status
Warning	● Low	RiskOracleConfig.sol	Resolved

Description

RiskOracleConfig should only expose setUint for the currently allowlisted risk-oracle parameters, because those parameters are consumed by the protocol as uint256 configs. Instead, the contract exposes generic setters for multiple DataStore types:

- setBool
- setAddress
- setBytes32
- setUint
- setInt

Because the same base-key allowlist is shared by all setters, a RISK_ORACLE account can accidentally call the wrong setter for a critical risk parameter. The transaction can succeed and emit a config event, but the actual uint config used by the protocol remains unchanged.

Recommendation

Remove the non-uint setters from RiskOracleConfig if the internal risk oracle only needs to update uint risk parameters.

Resolution

GMX Team: Resolved.

L-04 | RISK_ORACLE Role Missing In Scripts

Category	Severity	Location	Status
Warning	● Low	GLOBAL	Resolved

Description

The RISK_ORACLE is missing in multiple scripts:

- validateRolesUtils only uses the Role library and not the new Role2 library
- printRoles lacks the role in knownRoles
- validateContractDeployment lacks the role in roleLabels
- validateDeploymentUtils does not include it in the external-role allowlist

Recommendation

Consider to add the RISK_ORACLE to scripts wherever it is needed.

Resolution

GMX Team: Resolved.

I-01 | Unused Imports

Category	Severity	Location	Status
Superfluous Code	● Info	RiskOracleConfig.sol	Resolved

Description

RiskOracleConfig.sol imports Keys2, Precision, Cast, and MarketUtils, but none of these are used.

Recommendation

Consider to remove unused code.

Resolution

GMX Team: Resolved.

I-02 | _validateKey Performs A Redundant Role Check

Category	Severity	Location	Status
Access Control	● Info	RiskOracleConfig.sol	Resolved

Description

All public risk-oracle setters are already protected by the `onlyRiskOracle` modifier. However, `_validateKey` checks the same role again:

```
    if (roleStore.hasRole(msg.sender, Role2.RISK_ORACLE)) {  
        ...  
        return;  
    }  
    revert Errors.InvalidBaseKey(baseKey);
```

This role check is redundant for the current setters. Additionally, if `_validateKey` is reused by a future function that is not protected by `onlyRiskOracle`, callers without the role would receive `InvalidBaseKey` instead of the more accurate `Unauthorized` error.

Recommendation

Remove the role check from `_validateKey` and rely on the external `onlyRiskOracle` modifier for authorization.

Resolution

GMX Team: Resolved.

Remediation Findings & Resolutions

ID	Title	Category	Severity	Status
M-01	OPTIMAL_USAGE_FACTOR Updates Do Not Settle Accrued Borrowing	Logical Error	● Medium	Resolved
I-01	Allowlist Differs From Previous ConfigSyncer	Warning	● Info	Acknowledged
I-02	Coarser Operational Controls Than ConfigSyncer	Best Practices	● Info	Acknowledged
I-03	Incorrect NatSpec	Documentation	● Info	Resolved
I-04	Unused EventUtils	Superfluous Code	● Info	Resolved
I-05	Stale updateMarketConfigUtils Script	Configuration	● Info	Resolved

M-01 | OPTIMAL_USAGE_FACTOR Updates Do Not Settle Accrued Borrowing

Category	Severity	Location	Status
Logical Error	● Medium	RiskOracleConfig.sol	Resolved

Description

The remediation for stale borrowing accrual only settles borrowing before updates to `BASE_BORROWING_FACTOR` and `ABOVE_OPTIMAL_USAGE_BORROWING_FACTOR`.

However, `RiskOracleConfig` also allows the `RISK_ORACLE` role to update `OPTIMAL_USAGE_FACTOR`, which is part of the kink borrowing-rate formula. When it is non-zero, `MarketUtils.getBorrowingFactorPerSecond` uses it to route into `getKinkBorrowingFactor`:

```
uint256 optimalUsageFactor = getOptimalUsageFactor(dataStore, market.marketToken,
isLong);
if (optimalUsageFactor != 0) {
    return getKinkBorrowingFactor(
        ...
    );
}
```

Inside `getKinkBorrowingFactor`, the value controls the kink threshold and the divisor used to calculate the above-optimal portion of the borrowing rate. Despite this, `_isRiskOracleBorrowingBaseKey` excludes `OPTIMAL_USAGE_FACTOR`. As a result, `_settleBorrowingIfRequired` returns without settling accrued borrowing before the new `OPTIMAL_USAGE_FACTOR` is stored. The next borrowing update can then apply the new kink threshold to the full elapsed stale interval since `cumulativeBorrowingFactorUpdatedAt`. This can overcharge or undercharge open positions for the stale interval and may affect position health and liquidation eligibility. The impact depends on market staleness, utilization, and the size of the optimal usage factor change.

Recommendation

Include `OPTIMAL_USAGE_FACTOR` in `_isRiskOracleBorrowingBaseKey`.

Resolution

GMX Team: Resolved.

I-01 | Allowlist Differs From Previous ConfigSyncer

Category	Severity	Location	Status
Warning	● Info	RiskOracleConfig,sol	Acknowledged

Description

The new RiskOracleConfig allowlist is not a direct replacement for the previous Chaos Labs ConfigSyncer risk-oracle surface.

ConfigSyncer can apply updates for the following base keys that RiskOracleConfig does not allow:

- SWAP_IMPACT_FACTOR
- SWAP_IMPACT_EXPONENT_FACTOR
- BORROWING_FACTOR
- BORROWING_EXPONENT_FACTOR
- RESERVE_FACTOR
- OPEN_INTEREST_RESERVE_FACTOR

Conversely, RiskOracleConfig adds support for some keys that ConfigSyncer did not support, including:

- MAX_POSITION_IMPACT_FACTOR
- GLV_MAX_MARKET_TOKEN_BALANCE_USD
- GLV_MAX_MARKET_TOKEN_BALANCE_AMOUNT
- FUNDING_FACTOR
- FUNDING_EXPONENT_FACTOR
- THRESHOLD_FOR_STABLE_FUNDING
- THRESHOLD_FOR_DECREASE_FUNDING
- MIN_COLLATERAL_FACTOR
- MIN_COLLATERAL_FACTOR_FOR_OPEN_INTEREST_MULTIPLIER
- MIN_COLLATERAL_FACTOR_FOR_LIQUIDATION

This may be intentional if the internal Risk Oracle is meant to manage a different parameter set than the Chaos Labs integration. If not, the migration could silently remove automated risk-oracle management for swap impact, legacy borrowing, and reserve-factor parameters.

Recommendation

Confirm whether the new allowlist intentionally drops the ConfigSyncer-only keys.

Resolution

GMX Team: Acknowledged.

I-02 | Coarser Operational Controls Than ConfigSyncer

Category	Severity	Location	Status
Best Practices	● Info	RiskOracleConfig.sol	Acknowledged

Description

The older Chaos Labs risk-oracle flow used ConfigSyncer as an adapter between the external Risk Oracle and GMX config writes. Before applying an update, ConfigSyncer checked multiple operational disable flags:

- global sync feature disable;
- per-market disable;
- per-parameter disable;
- per-market-parameter disable.

RiskOracleConfig uses a simpler direct-write model. A CONFIG_KEEPER can enable or disable all risk-oracle updates for a market through setRiskOracleMarketEnabled, and a RISK_ORACLE account can then write allowlisted parameters for enabled markets.

This is simpler and may be intentional, but it is also less granular. If the internal risk oracle has an issue with one parameter family, such as funding, borrowing, or caps, operators can only disable the whole market or revoke the RISK_ORACLE role. They cannot pause a single base key or a single market/base-key pair while allowing other risk-oracle updates to continue.

Recommendation

Consider whether RiskOracleConfig should include finer emergency controls similar to ConfigSyncer, especially:

- a global risk-oracle config feature disable;
- a per-base-key disable;
- optionally, a per-market/base-key disable.

If this coarser control model is intentional, document the operational runbook for pausing faulty risk-oracle updates, including whether operators should disable affected markets or revoke the RISK_ORACLE role.

Resolution

GMX Team: Acknowledged.

I-03 | Incorrect NatSpec

Category	Severity	Location	Status
Documentation	● Info	RiskOracleConfig.sol: 214-228	Resolved

Description

The NatSpec of the `_validateKeyAndMarket` function states out that the function returns an address but it returns a market now.

Recommendation

Consider to update the NatSpec.

Resolution

GMX Team: Resolved.

I-04 | Unused EventUtils

Category	Severity	Location	Status
Superfluous Code	● Info	RiskOracleConfig.sol: 25	Resolved

Description

EventUtils.IntItems is unused now.

Recommendation

Consider to remove unused code.

Resolution

GMX Team: Resolved.

I-05 | Stale updateMarketConfigUtils Script

Category	Severity	Location	Status
Configuration	● Info	updateMarketConfigUtils.ts: 8-12	Resolved

Description

The list of RISK_ORACLE managed keys in the `updateMarketConfigUtils.ts` script is stale.

Recommendation

Consider to update this script.

Resolution

GMX Team: Resolved.

Disclaimer

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian’s position is that each company and individual are responsible for their own due diligence and continuous security. Guardian’s goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract’s safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

To learn more, visit <https://guardianaudits.com>

To view our audit portfolio, visit <https://github.com/guardianaudits>

To book an audit, message <https://t.me/guardianaudits>